Batch: B2 Roll No.: 16010124114

Experiment No.____3___

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of the Staff In-charge with date

Title: Study, Implementation, and Comparative Analysis of Merge Sort and Quick Sort.

Objective: To learn the divide and conquer strategy of solving the problems of different types

CO to be achieved:

CO 2 Describe various algorithm design strategies to solve different problems and analyze Complexity.

Books/ Journals/ Websites referred:

1. **Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press**
2. **T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algorithms",2nd Edition ,MIT press/McGraw Hill,2001**
3. **http://en.wikipedia.org/wiki/Quicksort**
4. **https://www.cs.auckland.ac.nz/~jmor159/PLDS210/qsort.html**
5. **http://www.cs.rochester.edu/~gildea/csc282/slides/C07-quicksort.pdf**
6. **http://www.sorting-algorithms.com/quick-sort**
7. **http://www.cse.ust.hk/~dekai/271/notes/L01a/quickSort.pdf**
8. **http://en.wikipedia.org/wiki/Merge_sort**
9. **http://www.personal.kent.edu/~rmuhamma/Algorithms/MyAlgorithms/Sorting/mergeSort.htm**
10. **http://www.sorting-algorithms.com/merge-sort**
11. **http://www.princeton.edu/~achaney/tmve/wiki100k/docs/Merge_sort.html**

Pre Lab/ Prior Concepts:

Data structures, various sorting techniques

Historical Profile:

Quicksort and merge sort are divide-and-conquer sorting algorithm in which division is dynamically carried out. They are one the most efficient sorting algorithms.

New Concepts to be learned:
Number of comparisons, Application of algorithmic design strategy to any problem, Classical problem solving vs Divide-and-Conquer problem solving.

Algorithm Recursive Quick Sort:

```
void quicksort( Integer A[ ], Integer left, Integer right)
//sorts A[left.. right] by using partition() to partition A[left.. right], and then //calling itself //
twice to sort the two subarrays.
{ IF ( left < right ) then
        {       q = partition( A, left, right);
                quicksort( A, left, q–1);
                quicksort( A, q+1, right);

        }
}
```

Integer *partition(integer A*T[], Integer *left*, Integer *right*)
//This function rarranges *A*[*left*..*right*] and finds and returns an integer *q*, such that *A*[*left*], ...,
//*A*[*q*–1] <~ *pivot*, *A*[*q*] = *pivot*, *A*[*q*+1], ..., *A*[*right*] > *pivot*, where *pivot* is the first element of //a[left…right], before partitioning.

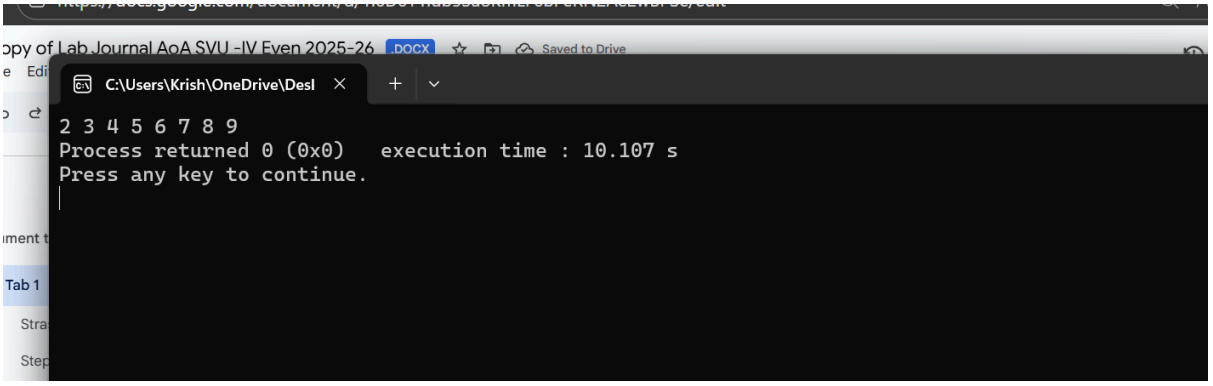
```
{
pivot = A[left]; lo = left+1; hi = right;
WHILE ( lo ≤ hi)
{       WHILE (A[hi] > pivot)                   hi = hi – 1;
        WHILE ( lo ≤ hi and A[lo] <~pivot)      lo = lo + 1;
        IF ( lo ≤ hi) then                      swap( A[lo], A[hi]);
}
swap(pivot, A[hi]);
  RETURN hi;
}
```

CODE:-

```
#include<bits/stdc++.h>

using namespace std;

int pivotfind(vector<int>&vc,int low,int high ){

  int pivott=vc[low];

  int lowest=low+1;

  int hightest=high;

  while(lowest<hightest){
```

```
  while(lowest<=high && vc[lowest]<=pivott ){

    lowest++;

  }

    while( hightest>=low && vc[hightest]>pivott){

    hightest--;

  }

  if(lowest<hightest){

  swap(vc[lowest],vc[hightest]);

  }

}

swap(vc[low],vc[hightest]);

  return hightest;

}


void quicksort(vector<int>&vc,int low,int high){

if(low>=high)return;

int pivot=pivotfind(vc,low,high);

quicksort(vc,low,pivot-1);

quicksort(vc,pivot+1,high);

}

int main(){

  vector<int>vc={9,3,7,5,6,4,8,2};

  int low=0;

  int high=vc.size()-1;

  quicksort(vc,low,high);

  for(int v:vc){
```

```
    cout<<v<<" ";

  }

}
```



The space complexity of Quick Sort:

Best case:-O(nlogn)
Worst case :-O(n^2)
Derivation of best case and worst-case time complexity (Quick Sort)

PAGE NO.:
DATE / /

Time complexity of Quick sort.

$T(n) = T(\text{size of left}) + T(\text{size of right}) + n$
$= T(n/2) + T(n/2) + n.$

$T(n) = 2T(n/2) + n$ —— (I)

$T(n/2 = 2T(n/4) + n/2.$

$T(n) = 4T(n/4) + 2n$ —— (II)

$T(n/4) = 2T(n/8) + n/4$
$= 8T(n/8) + 3n$ —— (III)

$T(n/k) = 2^k T(n/2^k) + 3kn.$

$\frac{n}{2^k} = 1$ $\quad n = 2^k$
$k = \log_2 n.$
$= O(n \log n)$

worst case $T(n) = T(n-1) + n$

$T(n-1) = T(n-2) + (n-1)$
$T(n) = T(n-2) + (n-1) + n.$
$T(n-2) = T(n-3) + n-2) + \ldots 2 + T(1)$
$T(n) = n(n+1)/2$
$T(n) = O(n^2)$

Algorithm Merge Sort
MERGE-SORT (A, p, r)

// To sort the entire sequence A[1 .. n], make the initial call to the procedure MERGE-SORT (A, //1, n). Array A and indices p, q, r such that $p \leq q \leq$ r and sub array $A[p .. q]$ is sorted and sub array //$A[q + 1 .. r]$ is sorted. By restrictions on p, q, r, neither sub array is empty.
//**OUTPUT**: The two sub arrays are merged into a single sorted sub array in $A[p .. r]$.

```
IF p < r                                  // Check for base case
   THEN q = FLOOR [(p + r)/2]             // Divide step
        MERGE (A, p, q)                   // Conquer step.
        MERGE (A, q + 1, r)               // Conquer step.
        MERGE (A, p, q, r)                // Conquer step.
```

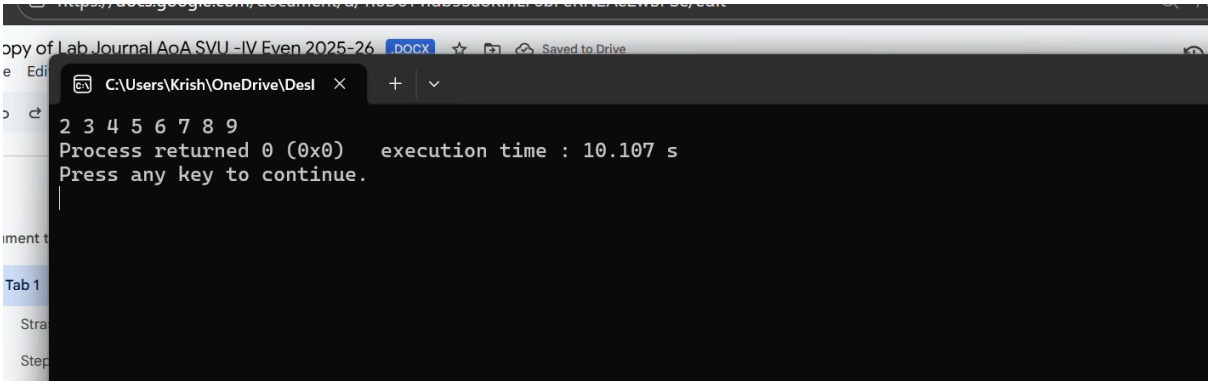
```
MERGE (A, p, q, r)
{
   n1 ← q − p + 1
   n2 ← r − q
   Create arrays L[1 . . n1 + 1] and R[1 . . n2 + 1]
   FOR i ← 1 TO n1
       DO L[i] ← A[p + i − 1]
   FOR j ← 1 TO n2
       DO R[j] ← A[q + j ]
   L[n1 + 1] ← ∞
   R[n2 + 1] ← ∞
  i ← 1
  j ← 1
  FOR k ← p TO r
     DO IF L[i ] ≤ R[ j]
          THEN A[k] ← L[i]
               i ← i + 1
          ELSE A[k] ← R[j]
               j ← j + 1

}
```

Code:-

```
#include<bits/stdc++.h>

using namespace std;

void merges(vector<int>&vc,int low,int middle,int high){

   vector<int>temp;

   int left=low;

   int right=middle+1;

   while(left<=middle && right<=high){

      if(vc[left]<vc[right]){

         temp.push_back(vc[left]);
```

```
            left++;
        }
        else{
            temp.push_back(vc[right]);
            right++;
        }
    }
    while(left<=middle){
        temp.push_back(vc[left]);
        left++;
    }
    while(right<=high){
        temp.push_back(vc[right]);
            right++;
    }
    for(int i=low;i<=high;i++){
        vc[i]=temp[i-low];
    }
}
void mergesort(vector<int>&vc,int low ,int high){
  if(low==high)return;
  int middle=(low+high)/2;
  mergesort(vc,low,middle);
  mergesort(vc,middle+1,high);
  merges(vc,low,middle,high);
}
```

```
int main(){
  vector<int>vc={9,3,7,5,6,4,8,2};
  int low=0;
  int high=vc.size()-1;
  mergesort(vc,low,high);
  for(int v:vc){
    cout<<v<<" ";
  }
}
```



The space complexity of Merge sort: O(nlogn) in all cases since extra array is required.

Derivation of best case and worst-case time complexity (Merge Sort)

$= O(\log n^n) = n\log n.$

Time Complexity of Merge sort.

divide.

- $T(n) = T(n/2) + T(n/2) + n$

  $T(n) = 2T(n/2) + n.$

  $T(n/2) = 2T(n/4) + n/2$

- $T(n) = 4T(n/4) + 2n$

  $T(n/4) = 4[T(n/8)] + n/4$

  $= 8T(n/8) + n + 2n$

- $T(n) = 8T(n/8) + 3n$

$T(n/k) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1$ $\quad n = 2^k$

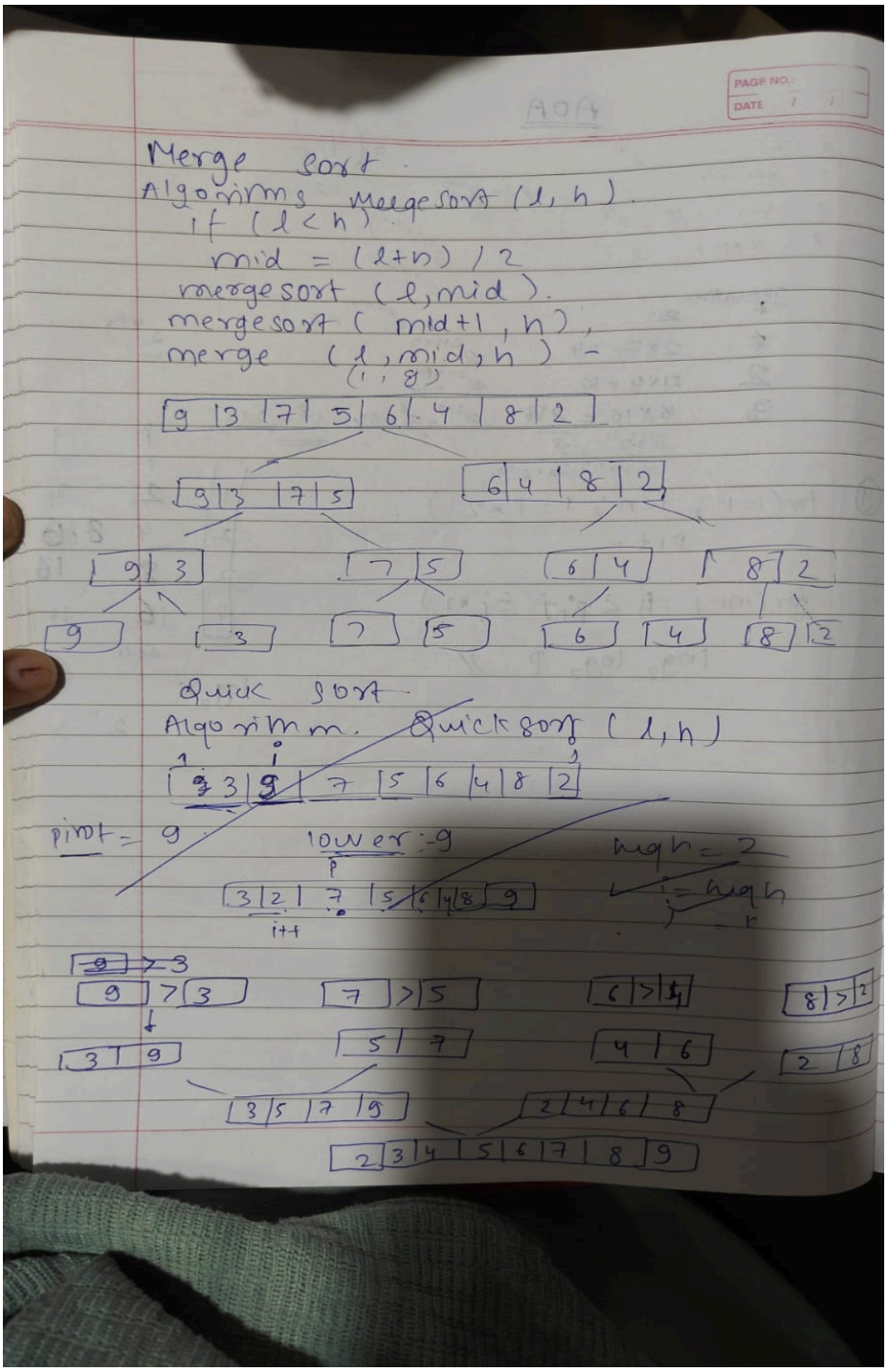
$\boxed{k = \log_2 n}$

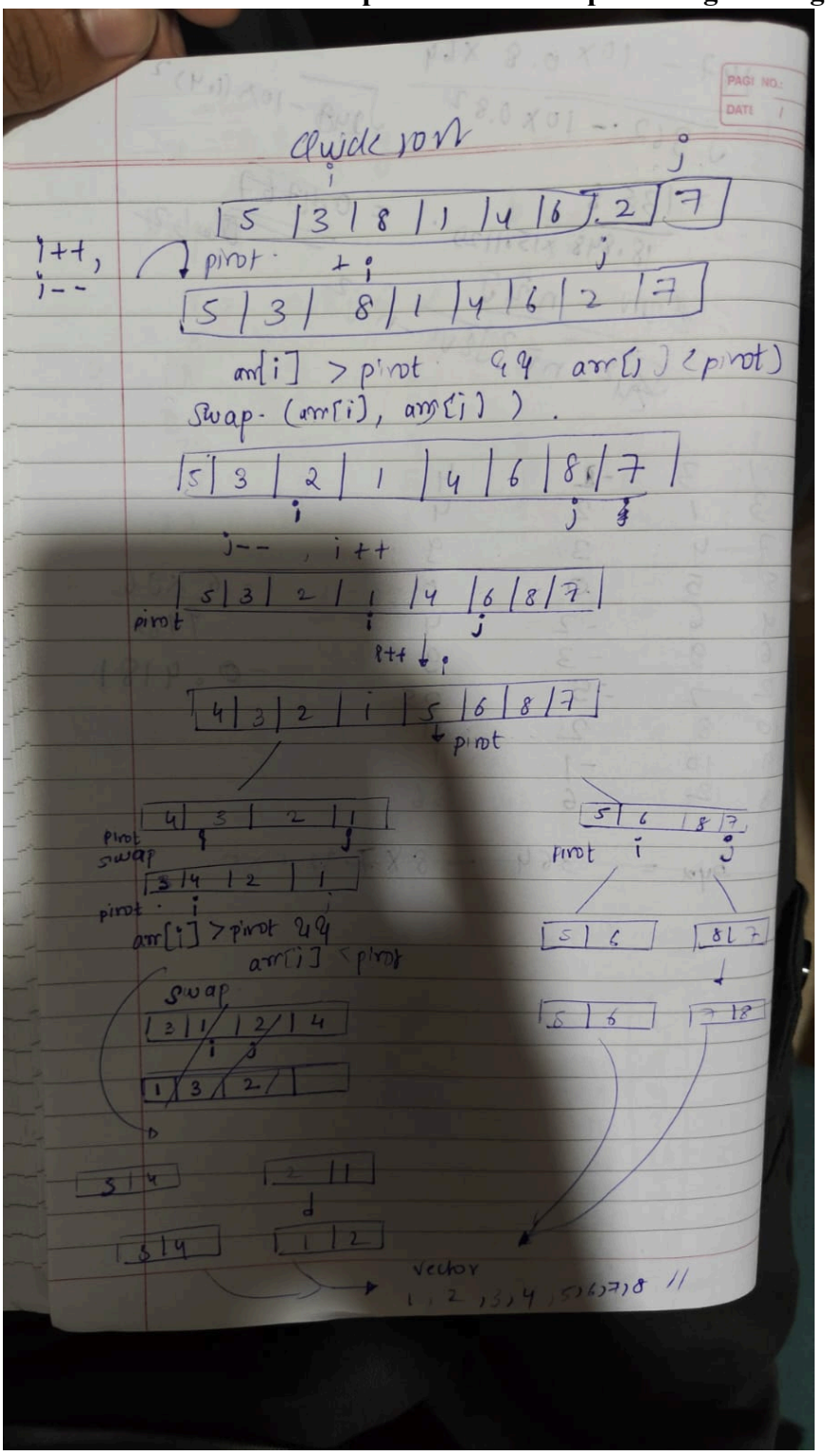
$T(n) = nT(1) + n\log n$

$= O(n\log n)$

$= O(n\log$

Example for quicksort/Merge tree for merge sort:





CONCLUSION: Successfully implemented merge sort and quick sort